AutoML-Lite

An Interactive Machine Learning Pipeline with a Streamlit GUI

Project Report

Prepared: June 2026

1. Introduction

AutoML-Lite is an end-to-end machine learning application that allows a user to upload any tabular dataset and obtain a trained, evaluated, and exportable model without writing a single line of code. The project combines a reusable scikit-learn pipeline with an interactive Streamlit-based graphical interface, making it accessible to both technical and non-technical users.

The motivation behind the project was to demonstrate a complete machine learning workflow — from raw data ingestion to model deployment-readiness — packaged inside a single, self-contained application suitable for a portfolio or as a reusable internal tool.

2. Objectives

- Build a reusable, generalizable ML pipeline that works on any tabular dataset, not a single fixed domain.
- Provide an intuitive GUI so that users without coding experience can train and compare models.
- Automatically detect whether a prediction task is classification or regression.
- Allow both automated multi-model comparison and manual single-model experimentation.
- Support common real-world data formats (CSV and Excel).
- Provide diagnostic visualizations and feature importance to make model behavior interpretable.
- Allow the trained pipeline to be exported and reused outside the application.

3. System Architecture

The project is structured into two clearly separated layers: the pipeline logic and the graphical interface. This separation keeps the machine learning code testable and reusable independently of Streamlit.

3.1 Project Structure

```
automl_lite/   app.py                 Streamlit GUI   pipeline/
__init__.py     core.py              Profiling, preprocessing, training,
evaluation   requirements.txt   README.md
```

3.2 Data Flow

- User uploads a CSV or Excel file (or selects a demo dataset) through the sidebar.
- The app profiles the dataset: row/column counts, missing values, data types, duplicate rows, and a correlation heatmap.
- The user selects a target column; the task type (classification or regression) is inferred automatically and can be overridden.
- The dataset is split into training and test sets using a user-adjustable test size.

- A preprocessing pipeline imputes missing values, scales numeric features, and one-hot encodes categorical features.
- One or more models are trained on the processed data, depending on which tab the user is in.
- Results are evaluated on the held-out test set and displayed as a ranked leaderboard with diagnostic charts.
- The trained pipeline (preprocessing + model bundled together) can be downloaded as a .pkl file.

4. Key Features

4.1 Data Ingestion

- Accepts CSV files as well as Excel files (.xlsx, .xls, .xlsm).
- Automatically detects multi-sheet Excel workbooks and lets the user choose which sheet to load.
- Includes built-in demo datasets (Titanic for classification, Tips for regression) for quick exploration.

4.2 Data Profiling

- Row and column counts, duplicate row detection.
- Per-column data type and missing-value percentage breakdown.
- Interactive correlation heatmap for numeric columns.

4.3 Three Training Modes

The application offers three distinct tabs to suit different levels of user control:

Tab	Description
Auto Leaderboard	Trains every available model for the detected task in one click and ranks them automatically on the held-out test set.
Classification	Lets the user pick one specific classification model, the target column, and the test size, then trains only that model.
Regression	Lets the user pick one specific regression model, the target column, and the test size, then trains only that model.

4.4 Diagnostics & Interpretability

- Confusion matrix heatmap for classification models.
- Residual plot and Actual-vs-Predicted scatter plot for regression models.
- Feature importance bar chart (tree-based importances or linear coefficients, depending on model type).

4.5 Model Export

The best (or selected) trained pipeline — including all preprocessing steps — can be downloaded as a single .pkl file using joblib, ready to be loaded and used for predictions on new data in any Python environment.

5. Supported Models

Classification	Regression
Logistic Regression	Linear Regression
Random Forest Classifier	Random Forest Regressor
Support Vector Machine (SVC)	Support Vector Regressor (SVR)
Decision Tree Classifier	Decision Tree Regressor
k-Nearest Neighbors (kNN)	—
XGBoost Classifier	XGBoost Regressor

Note: k-means clustering was considered but intentionally excluded from the supervised leaderboard, since it is an unsupervised algorithm and does not produce predictions that can be scored against a labeled target using accuracy, F1, or R^2. It would require a separate, dedicated clustering exploration view.

6. Evaluation Metrics

6.1 Classification

- Accuracy — proportion of correct predictions.
- F1 Score (weighted) — harmonic mean of precision and recall, used as the primary ranking metric.
- Precision (weighted) — proportion of positive predictions that were correct.
- Recall (weighted) — proportion of actual positives that were correctly identified.

6.2 Regression

- RMSE (Root Mean Squared Error) — average magnitude of prediction error, in the target's original units.
- MAE (Mean Absolute Error) — average absolute prediction error.
- R^2 Score — proportion of variance in the target explained by the model, used as the primary ranking metric.

7. Technology Stack

Component	Technology
GUI / Frontend	Streamlit
ML Modeling	scikit-learn, XGBoost

Data Handling	pandas, NumPy, openpyxl, xlrd
Visualization	Plotly
Model Persistence	joblib
Language	Python 3.10+

8. Challenges & Solutions

Challenge	Solution
XGBoost requires numeric (0..n-1) class labels, but real-world targets are often strings.	Used a LabelEncoder specifically for the XGBoost classifier path, encoding before fit and decoding predictions before scoring.
Different model types expose feature importance differently (tree-based vs. linear).	Built a unified get_feature_importance() helper that checks for feature_importances_ or coef_ and falls back gracefully if neither exists (e.g. SVM).
Supporting both automated multi-model comparison and manual single-model selection without duplicating UI code.	Added an optional model_names filter to the core training function and a shared render_results() helper reused across all three Streamlit tabs.
Excel files may contain multiple sheets.	Used pandas.ExcelFile to list sheet names and dynamically show a sheet-selection dropdown only when needed.

9. Future Scope

- Hyperparameter tuning (GridSearchCV / Optuna) with a live progress indicator.
- SHAP-based explainability for richer, per-prediction interpretability.
- Cross-validation instead of a single train/test split for more robust evaluation.
- A dedicated unsupervised tab for clustering (e.g. k-means) and dimensionality reduction (PCA).
- Time-series-aware train/test splitting for temporal datasets.
- Containerization with Docker and deployment to Streamlit Community Cloud.

10. Conclusion

AutoML-Lite demonstrates a complete, practical machine learning workflow wrapped in an accessible graphical interface. By separating the pipeline logic from the GUI, the project remains maintainable and extensible, while the three distinct training modes give users the flexibility to either explore broadly with an automated leaderboard or experiment precisely with a single chosen model. The project illustrates core data science skills — data profiling,

preprocessing, model selection, evaluation, interpretability, and deployment readiness — in a single cohesive, reusable application.